Modelo de Difusão DDPM para Geração de Faces

Arquitetura, Treinamento e Amostragem

Lorran de Araújo Durães Soares

Tópicos Avançados em Redes Neurais Profundas

25 de agosto de 2026

Objetivo

- Treinar um modelo generativo capaz de produzir imagens de faces a partir de ruído aleatório.
- O modelo usado é um DDPM: *Denoising Diffusion Probabilistic Model* [1].
- A rede neural principal é uma UNet convolucional condicionada pelo passo temporal de difusão [2].
- A geração é feita por remoção progressiva de ruído, do passo T até o passo 0.

Fluxo Geral do Projeto

1. Normalizar as imagens para o intervalo $[-1, 1]$.
2. Durante o treinamento, adicionar ruído gaussiano às imagens.
3. Treinar uma UNet para prever o ruído adicionado.
4. Durante a geração, começar de ruído puro.
5. Aplicar o processo reverso até obter uma imagem sintética.

Dataset e Pré-processamento

- Dataset usado: subconjunto do UTKFace [10].
- Filtro aplicado: faces femininas entre 20 e 40 anos.
- Imagens convertidas para RGB e redimensionadas para 128×128.
- Normalização:

$$x \in [0, 1] \rightarrow x \in [-1, 1]$$

- Aumento de dados: `RandomHorizontalFlip` com probabilidade 0.5.

Ideia Central do DDPM

Processo direto

Uma imagem real é corrompida progressivamente com ruído gaussiano.

Processo reverso

O modelo aprende a remover esse ruído passo a passo.

- No treinamento, conhecemos a imagem limpa e o ruído aplicado.
- A rede aprende a prever o ruído presente em uma imagem ruidosa.
- Na geração, começamos de ruído puro e usamos a rede para reverter o processo.

Processo Direto: Adição de Ruído

O scheduler define uma sequência linear de variâncias β_t [1].

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

A imagem ruidosa em um passo t é obtida por:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

onde $\epsilon \sim \mathcal{N}(0, I)$.

Objetivo de Treinamento

A rede neural recebe:

$$(x_t, t)$$

e prediz:

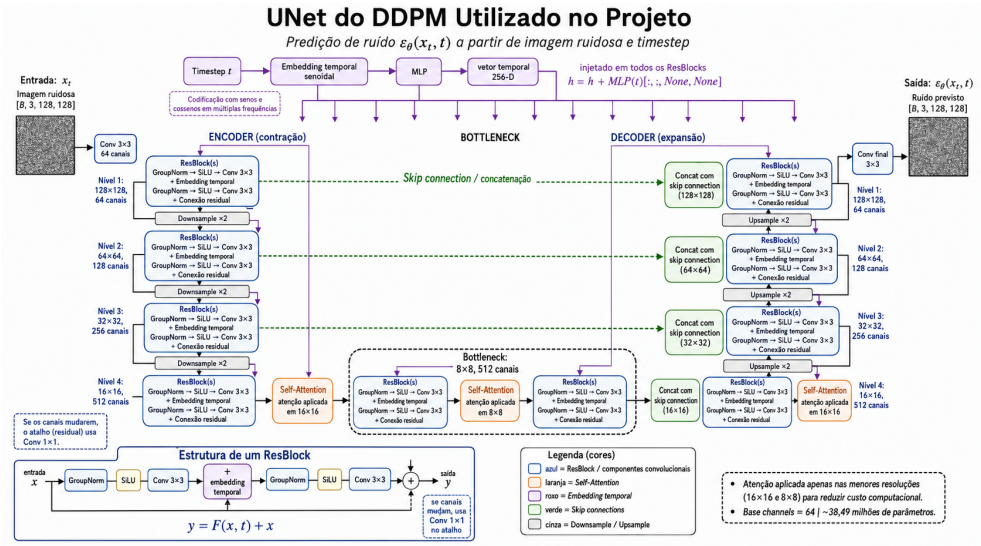
$$\epsilon_\theta(x_t, t)$$

A função de perda é [1]:

$$\mathcal{L} = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]$$

Ou seja, o modelo não prediz diretamente a imagem final; ele prediz o ruído.

Arquitetura Geral da UNet I



Arquitetura Geral da UNet II

- Entrada: imagem ruidosa x_t com forma $[B, 3, 128, 128]$.
- Condicionamento: timestep t via embedding temporal.
- Saída: ruído previsto com a mesma forma da imagem.
- Estrutura encoder-decoder com conexões de atalho [2].
- Encoder captura contexto global.
- Decoder reconstrói a resolução espacial.
- Skip connections preservam detalhes locais.

Escalas da UNet

Etapa	Resolução	Canais
Entrada	128×128	3
Nível 1	128×128	64
Nível 2	64×64	128
Nível 3	32×32	256
Nível 4	16×16	512
Bottleneck	8×8	512
Saída	128×128	3

Com `base_channels = 64`, o modelo possui aproximadamente 38.49 milhões de parâmetros.

Bloco Residual

Cada `ResBlock` contém:

- GroupNorm [5].
- Ativação SiLU.
- Convolução 3×3.
- Injeção do embedding temporal.
- Nova sequência GroupNorm, SiLU e convolução 3×3.
- Conexão residual entre entrada e saída [3].

$$y = F(x, t) + x$$

Se o número de canais muda, usa-se uma convolução 1×1 no caminho residual.

Embedding Temporal

O modelo precisa saber o nível de ruído de cada amostra.

- O timestep t é codificado com senos e cossenos em múltiplas frequências.
- Essa estratégia é inspirada nos embeddings posicionais dos Transformers [4].
- Depois, um MLP projeta o embedding para dimensão 256.
- O embedding é somado aos mapas de características dentro dos ResBlocks.

Injeção temporal no bloco:

$$h = h + \text{MLP}(t)[:, :, None, None]$$

Self-Attention

- A atenção permite que regiões distantes da imagem interajam.
- Isso é útil para faces, pois olhos, boca, cabelo e contorno precisam ser globalmente coerentes.
- O modelo usa atenção nas resoluções menores: 16×16 e 8×8.
- Aplicar atenção em 128×128 seria muito custoso em memória.

Atenção por produto escalar [4]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{C}}\right) V$$

Treinamento do DDPM

Algorithm 1 Treinamento do modelo DDPM

Require: Dataset de imagens D, número de timesteps T, UNet ϵ_θ, scheduler de ruído

for cada época **do**
  for cada batch de imagens $x_0 \sim D$ **do**
    Amostrar timestep
    $$t \sim \mathcal{U}\{0, \ldots, T-1\}$$
    Amostrar ruído gaussiano
    $$\epsilon \sim \mathcal{N}(0, I)$$
    Gerar imagem ruidosa
    $$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$$
    Predizer o ruído com a UNet
    $$\hat{\epsilon} = \epsilon_\theta(x_t, t)$$
    Calcular a perda
    $$\mathcal{L} = \|\epsilon - \hat{\epsilon}\|_2^2$$
    Atualizar θ usando backpropagation e AdamW
    Aplicar gradient clipping
    Atualizar os pesos EMA
  end for
end for

Loop de Treinamento

Para cada batch de imagens reais x_0:

1. Sorteia-se um timestep t para cada imagem.
2. Sorteia-se ruído gaussiano ϵ.
3. Calcula-se x_t usando o scheduler.
4. A UNet prediz $\epsilon_\theta(x_t, t)$.
5. Calcula-se a loss MSE.
6. Executa-se backpropagation.
7. Aplica-se gradient clipping.
8. Atualiza-se o modelo com AdamW [6].
9. Atualiza-se a EMA dos pesos.

Hiperparâmetros Principais

Hiperparâmetro	Valor atual
Tamanho da imagem	128
Batch size	16
Épocas	200
Learning rate	0.0002
Timesteps	1000
Base channels	64
EMA decay	0.999
Gradient clipping	1.0
Augmentation	Horizontal flip

EMA: Média Móvel Exponencial

A EMA mantém uma segunda cópia do modelo, suavizando os pesos ao longo do treinamento.

$$\theta_{EMA} \leftarrow \lambda\theta_{EMA} + (1 - \lambda)\theta$$

No projeto:

$$\lambda = 0.999$$

Logo:

$$\theta_{EMA} \leftarrow 0.999\theta_{EMA} + 0.001\theta$$

Por que Usar EMA?

- Os pesos do modelo principal oscilam durante o treinamento.
- A oscilação vem dos batches aleatórios, timesteps aleatórios e ruído gaussiano.
- A EMA cria uma versão temporalmente suavizada dos pesos [7, 8].
- Em modelos generativos, a EMA costuma gerar imagens mais estáveis e coerentes.
- A EMA não altera a loss: o modelo principal continua treinando com MSE.

Processo de Geração I

Algorithm 2 Processo reverso de geração

Require: UNet treinada ϵ_θ, número de timesteps T

Inicializar com ruído puro

$$x_T \sim \mathcal{N}(0, I)$$

for $t = T-1, T-2, \ldots, 0$ **do**

  Predizer o ruído presente em x_t

$$\hat{\epsilon} = \epsilon_\theta(x_t, t)$$

  Calcular a média da distribuição reversa

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\epsilon} \right)$$

  if $t > 0$ **then**

    Amostrar

$$z \sim \mathcal{N}(0, I)$$

    Atualizar

$$x_{t-1} = \mu_\theta(x_t, t) + \sqrt{\tilde{\beta}_t}\, z$$

  else

$$x_0 = \mu_\theta(x_t, t)$$

  end if

end for

return imagem gerada x_0

Processo de Geração II

A geração começa com ruído puro:

$$x_T \sim \mathcal{N}(0, I)$$

Depois, para cada timestep t de $T-1$ até 0:

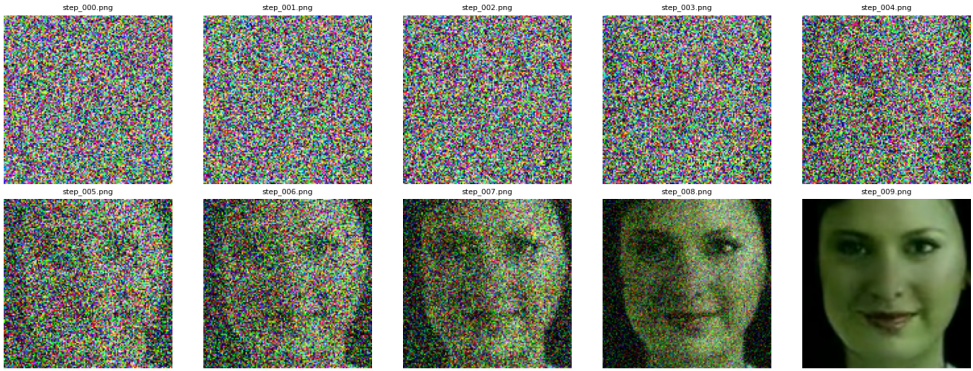
1. A UNet prediz o ruído presente em x_t.
2. Calcula-se a média da distribuição reversa.
3. Se $t > 0$, adiciona-se ruído com variância posterior.
4. Se $t = 0$, a média é usada como imagem final.

Exemplos de imagens geradas



Visualização do processo de Denoising

Processo de Denoising



Limitações Atuais

- A geração DDPM com 1000 passos é lenta.
- O modelo é incondicional: não controla idade, gênero ou etnia explicitamente.
- Não há métrica quantitativa como FID implementada.
- O scheduler de ruído é linear.
- O conjunto filtrado tem tamanho limitado para geração de faces.

Possíveis Melhorias

- Implementar DDIM para amostragem mais rápida [9].
- Testar cosine noise schedule.
- Usar mixed precision para acelerar o treinamento.
- Adicionar condicionamento por atributos do UTKFace.
- Medir qualidade com FID.
- Treinar por mais tempo ou usar dataset maior.

Resumo

- O DDPM aprende a inverter um processo de adição gradual de ruído.
- A UNet prediz o ruído em cada timestep.
- A arquitetura usa múltiplas escalas, ResBlocks, skip connections e atenção [2, 3, 4].
- O treinamento usa MSE, AdamW, gradient clipping e EMA.
- Na geração, a versão EMA do modelo é preferida por produzir amostras mais estáveis.

Referências

- Ho, J.; Jain, A.; Abbeel, P. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.
- Ronneberger, O.; Fischer, P.; Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015.
- He, K.; Zhang, X.; Ren, S.; Sun, J. *Deep Residual Learning for Image Recognition*. CVPR, 2016.
- Vaswani, A. et al. *Attention Is All You Need*. NeurIPS, 2017.
- Wu, Y.; He, K. *Group Normalization*. ECCV, 2018.
- Loshchilov, I.; Hutter, F. *Decoupled Weight Decay Regularization*. ICLR, 2019.
- Nichol, A. Q.; Dhariwal, P. *Improved Denoising Diffusion Probabilistic Models*. ICML, 2021.
- Dhariwal, P.; Nichol, A. Q. *Diffusion Models Beat GANs on Image Synthesis*. NeurIPS, 2021.
- Song, J.; Meng, C.; Ermon, S. *Denoising Diffusion Implicit Models*. ICLR, 2021.
- Zhang, Z.; Song, Y.; Qi, H. *Age Progression/Regression by Conditional Adversarial Autoencoder*. CVPR, 2017.